

Diseño Técnico del Proyecto

Texticode — Sistema de Gestión Textil
API de Carga de Trabajo

Aprendices

Kevin Stevan Wagner
Kevin Andrés Castro
Andrés Felipe Lancheros
Camilo Andrés Tibambre

Instructor

Ing. Edwin Marín

Fecha

Mayo 2026

Programa

ADSO 2026

1. Arquitectura 3 Capas

Texticode implementa una arquitectura REST desacoplada en 3 capas. El frontend nunca accede directamente a la base de datos — toda comunicación ocurre vía HTTP con autenticación por API Key.

FRONTEND
Vue.js 3 · EficienciaOperarios.vue · Fetch HTTP · VITE_API_KEY en .env

v HTTP / REST (x-api-key)

BACKEND
Node.js + Express · routes/carga_trabajo.js · middleware apiKey.js · Logica de negocio

v SQL queries (pg.Pool — Supabase Session Pooler 6543)

BASE DE DATOS
Supabase PostgreSQL · tablas: usuario, rol, orden_produccion

FLUJO DE DATOS — DETECTAR SOBRECARGA Y REASIGNAR ÓRDENES

Capa	Acción
Frontend	EficienciaOperarios.vue → fetch('/api/carga-trabajo', { headers: { 'x-api-key': KEY } })
Backend Auth	Middleware apiKey.js verifica x-api-key contra API_KEY_EFICIENCIA en .env
Backend Business	carga_trabajo.js clasifica operarios: sobrecargado (>5 ordenes) / normal (3-5) / disponible (<=2)
Persistence	pg.Pool ejecuta SELECT + JOIN en Supabase PostgreSQL (Session Pooler 6543)
Database	PostgreSQL devuelve registros con garantías ACID
Response	200 OK + { ok: true, resumen: { sobrecargados, normales, disponibles }, data: [...] }

2. Contrato Técnico API

Los 9 contratos de la API de Carga de Trabajo documentados con evidencia real de pruebas en Thunder Client.

2.1 GET /api/carga-trabajo

GET	/api/carga-trabajo <i>Lista todos los operarios activos clasificados por carga en tiempo real.</i>
-----	--

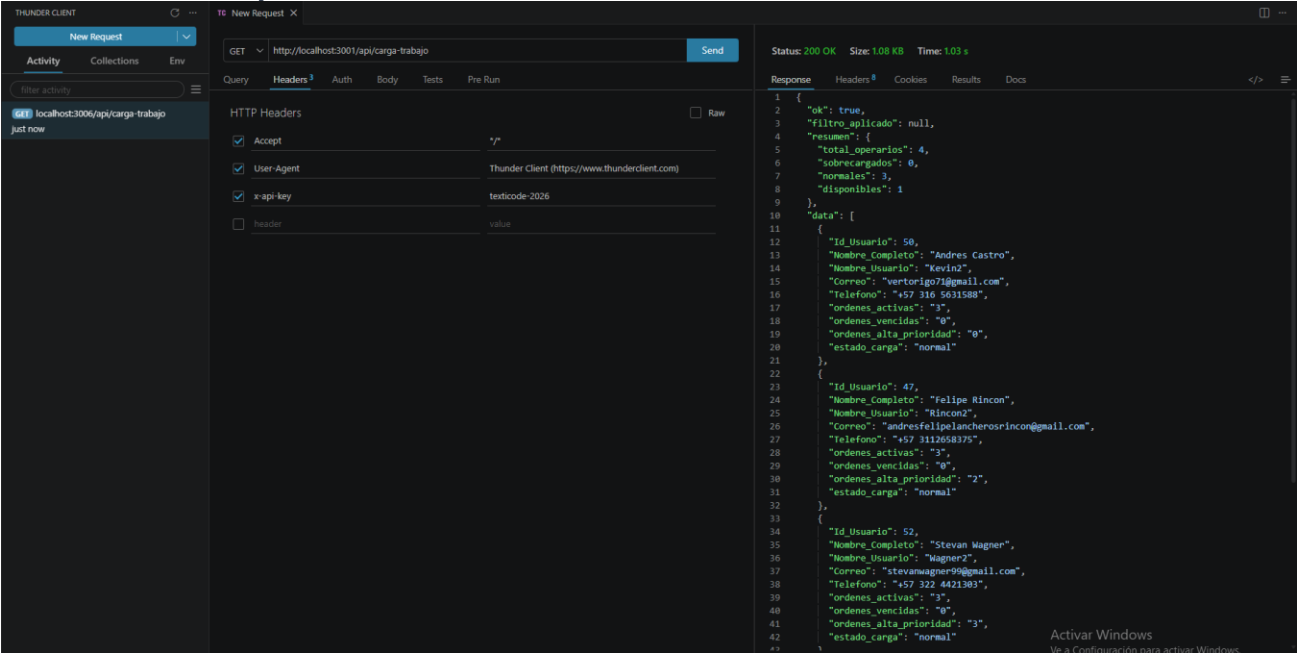
REQUEST

```
GET /api/carga-trabajo HTTP/1.1
Host: localhost:3006
x-api-key: texticode-2026

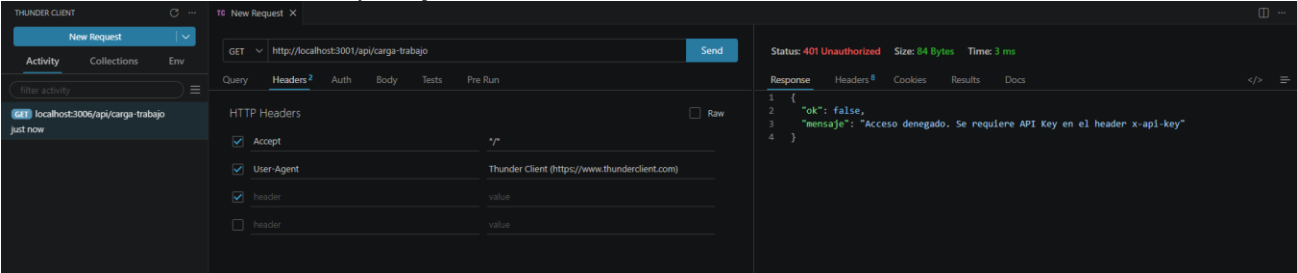
// Query params opcionales
?estado=sobrecargado|normal|disponible
```

EVIDENCIA THUNDER CLIENT

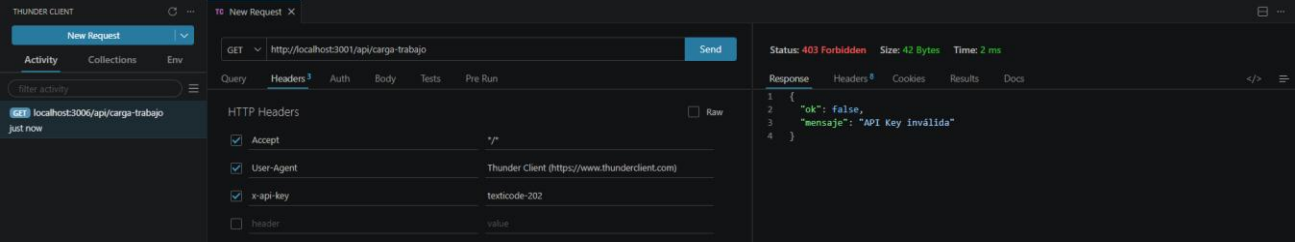
Prueba 1 — API Key correcta → 200 OK



Prueba 2 — Sin header x-api-key → 401 Unauthorized



Prueba 3 — API Key incorrecta → 403 Forbidden



2.2 GET /api/carga-trabajo/sugerencias

GET	/api/carga-trabajo/sugerencias <i>Genera sugerencias automaticas de reasignacion priorizando ordenes vencidas y de alta prioridad.</i>
-----	--

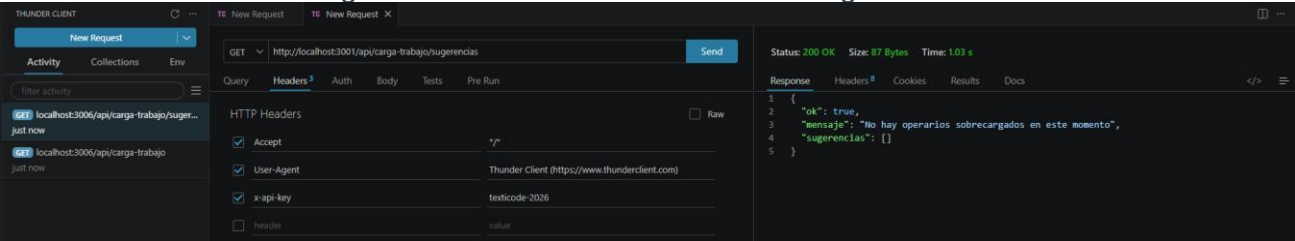
REQUEST

```
GET /api/carga-trabajo/sugerencias HTTP/1.1
Host: localhost:3006
x-api-key: texticode-2026

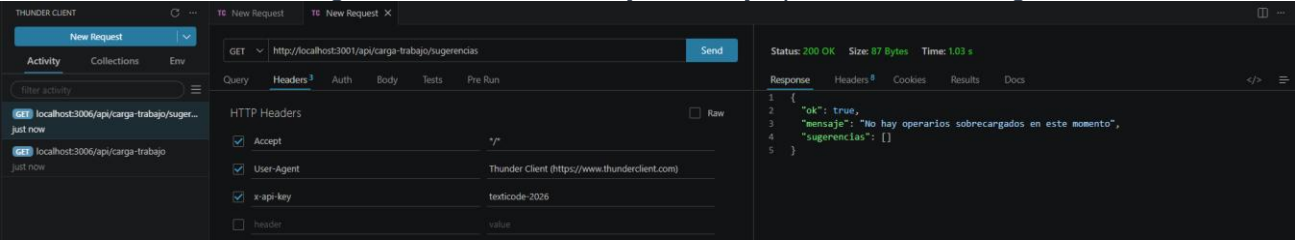
// Sin body ni parametros - calcula automaticamente
```

EVIDENCIA THUNDER CLIENT

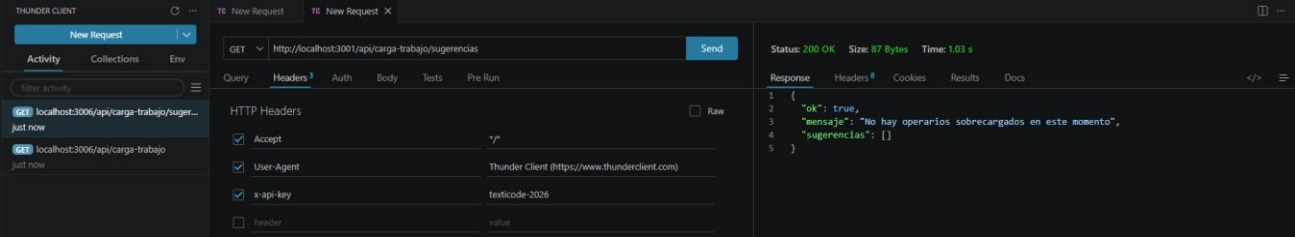
Prueba 1 — Con sobrecarga activa → 200 OK + movimientos sugeridos



Prueba 2 — Sin sobrecarga → 200 OK + mensaje 'No hay operarios sobrecargados'



Prueba 3 — Sin disponibles → 200 OK + mensaje 'Ninguno tiene capacidad disponible'



2.3 GET /api/carga-trabajo/operarios/:id

GET

/api/carga-trabajo/operarios/:id

Devuelve detalle completo de carga de un operario especifico con sus ordenes activas.

REQUEST

```
GET /api/carga-trabajo/operarios/28 HTTP/1.1
Host: localhost:3006
x-api-key: texticode-2026

// :id -> Id_Usuario del operario (entero positivo)
```

EVIDENCIA THUNDER CLIENT

Prueba 1 — Operario existente (id: 43) → 200 OK

The screenshot shows the Thunder Client interface with a GET request to `http://localhost:3001/api/carga-trabajo/operarios/43`. The request headers include `Accept: */*`, `User-Agent: Thunder Client (https://www.thunderclient.com)`, and `x-api-key: texticode-2026`. The response status is `200 OK` with a size of `478 Bytes` and a time of `1.01 s`. The response body is a JSON object containing user details and active orders.

```
1 {
2   "ok": true,
3   "data": {
4     "id_usuario": 43,
5     "Nombre Completo": "Andres Montaña ",
6     "Nombre Usuario": "Montaña2",
7     "Correo": "uscamliotibambre@gmail.com",
8     "Telefono": "457 320 3590156",
9     "ordenes_activas": "1",
10    "ordenes_vencidas": "0",
11    "ordenes_alta_prioridad": "0",
12    "estado_carga": "disponible",
13    "ordenes_activas_detalle": [
14      {
15        "id_Orden": 134,
16        "Producto": "Camisas ",
17        "Estado": "En Proceso",
18        "Prioridad": "Media",
19        "Unidades": 100,
20        "Unidades Realizadas": 20,
21        "fecha_limite": "2026-05-31T05:00:00.000Z",
22        "vencida": false
23      }
24    ]
25  }
26 }
```

Prueba 2 — Operario inexistente (id: 9999) → 404 Not Found

The screenshot shows the Thunder Client interface with a GET request to `http://localhost:3001/api/carga-trabajo/operarios/28`. The response status is `404 Not Found` with a size of `59 Bytes` and a time of `912 ms`. The response body is a JSON object indicating that the operator was not found.

```
1 {
2   "ok": false,
3   "mensaje": "No se encontró operario con id 28"
4 }
```

Prueba 3 — ID no numerico (id: abc) → 400 Bad Request

The screenshot shows the Thunder Client interface with a GET request to `http://localhost:3001/api/carga-trabajo/operarios/id`. The response status is `400 Bad Request` with a size of `58 Bytes` and a time of `1 ms`. The response body is a JSON object indicating that the ID is not a valid number.

```
1 {
2   "ok": false,
3   "mensaje": "El id debe ser un número válido"
4 }
```

2.4 POST /api/carga-trabajo/reasignar

POST

/api/carga-trabajo/reasignar

Reasigna una orden de produccion de un operario a otro.

REQUEST

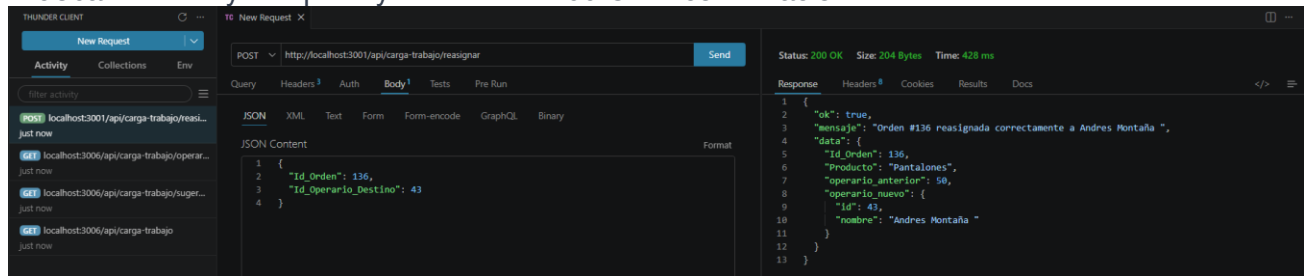
```
POST /api/carga-trabajo/reasignar HTTP/1.1
Host: localhost:3006
x-api-key: texticode-2026
Content-Type: application/json

{
  "Id_Orden": 13,
  "Id_Operario_Destino": 27
}

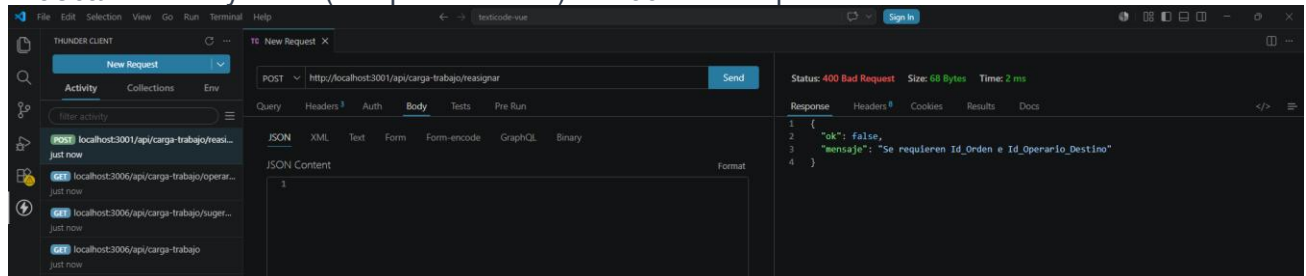
// Ambos campos son requeridos.
// La orden debe estar En Proceso o Pausado.
// El operario destino debe ser operario activo.
```

EVIDENCIA THUNDER CLIENT

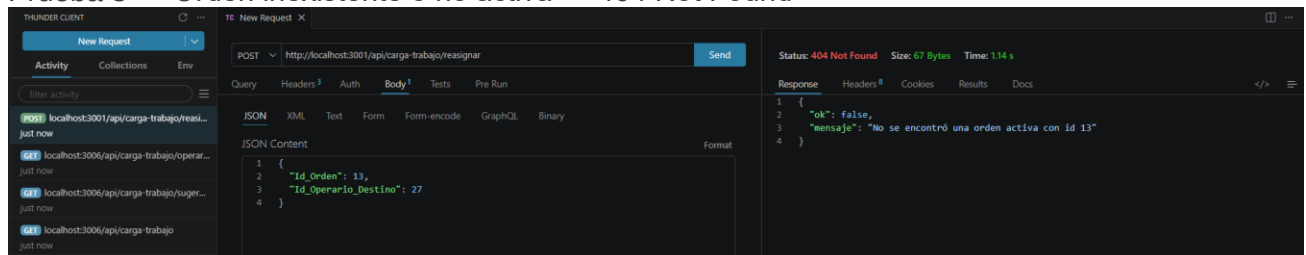
Prueba 1 — Body completo y correcto → 200 OK + confirmación



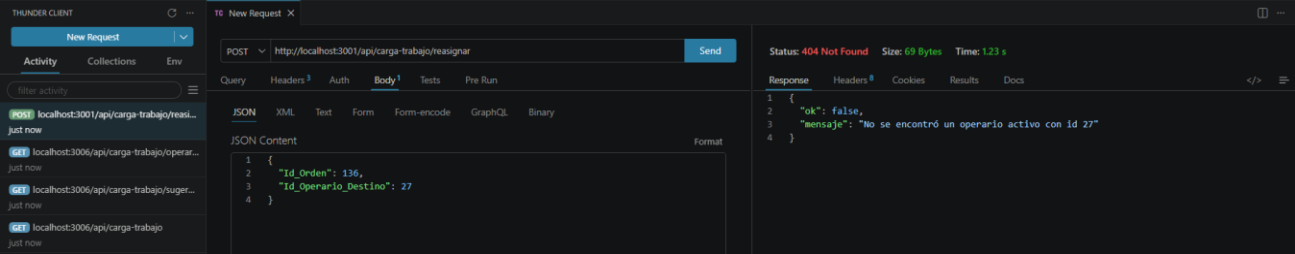
Prueba 2 — Body vacio (campos faltantes) → 400 Bad Request



Prueba 3 — Orden inexistente o no activa → 404 Not Found



Prueba 4 — Operario destino inexistente → 404 Not Found



2.5 POST /api/carga-trabajo/reasignar-multiple

POST	/api/carga-trabajo/reasignar-multiple Reasigna varias ordenes en una sola llamada. Procesa cada movimiento de forma independiente.
------	---

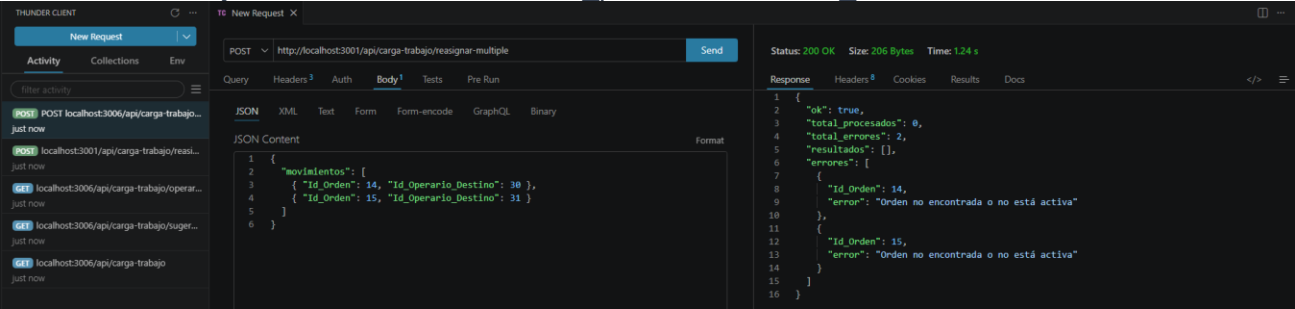
REQUEST

```
POST /api/carga-trabajo/reasignar-multiple HTTP/1.1
Host: localhost:3006
x-api-key: texticode-2026
Content-Type: application/json

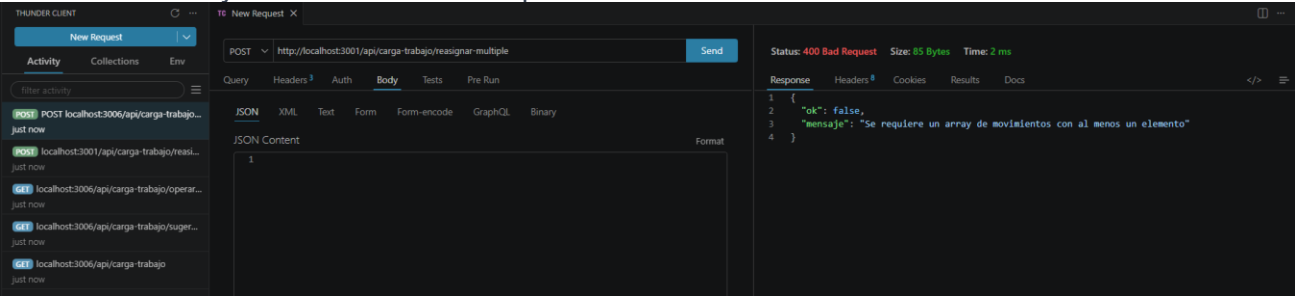
{
  "movimientos": [
    { "Id_Orden": 13, "Id_Operario_Destino": 27 },
    { "Id_Orden": 14, "Id_Operario_Destino": 30 },
    { "Id_Orden": 15, "Id_Operario_Destino": 31 }
  ]
}
```

EVIDENCIA THUNDER CLIENT

Prueba 1 — Array correcto → 200 OK + total procesados + total errores



Prueba 2 — Array vacio → 400 Bad Request



2.6 POST /api/carga-trabajo/aplicar-sugerencias

POST	/api/carga-trabajo/aplicar-sugerencias <i>Calcula y ejecuta automaticamente todas las reasignaciones sugeridas sin intervencion manual.</i>
------	---

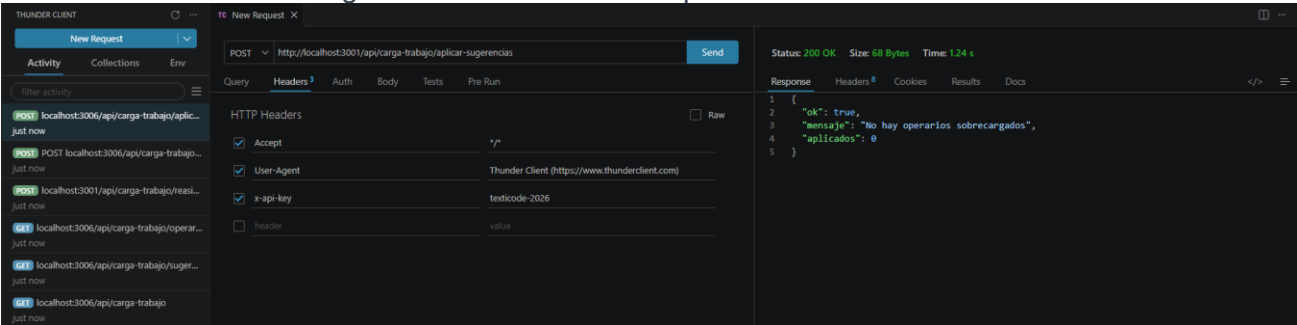
REQUEST

```
POST /api/carga-trabajo/aplicar-sugerencias HTTP/1.1
Host: localhost:3006
x-api-key: texticode-2026

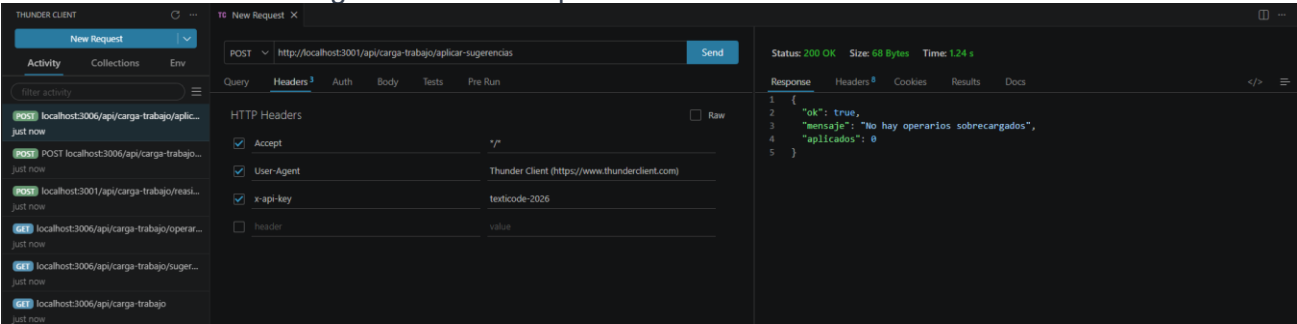
// Sin body - calcula y aplica automaticamente
```

EVIDENCIA THUNDER CLIENT

Prueba 1 — Con sobrecarga activa → 200 OK + N aplicados + detalle de movimientos



Prueba 2 — Sin sobrecarga → 200 OK + aplicados: 0



2.7 PATCH /api/carga-trabajo/umbrales

PATCH

/api/carga-trabajo/umbrales

Cambia los limites de sobrecarga y disponibilidad en caliente sin reiniciar el servidor.

REQUEST

```
PATCH /api/carga-trabajo/umbrales HTTP/1.1
Host: localhost:3006
x-api-key: texticode-2026
Content-Type: application/json

{
  "limite_sobrecarga": 3,
  "limite_disponible": 1
}

// Ambos campos son opcionales - se puede enviar solo uno.
// limite_sobrecarga: entero >= 1
// limite_disponible: entero >= 0
```

EVIDENCIA THUNDER CLIENT

Prueba 1 — Body correcto → 200 OK + umbrales actualizados

Prueba 2 — limite_sobrecarga = 0 → 400 Bad Request

2.8 PATCH /api/carga-trabajo/ordenes/:id/operario

PATCH

/api/carga-trabajo/ordenes/:id/operario

Cambia el operario asignado a una orden especifica.

REQUEST

```
PATCH /api/carga-trabajo/ordenes/14/operario HTTP/1.1
Host: localhost:3006
x-api-key: texticode-2026
Content-Type: application/json

{
  "Id_Operario_Destino": 30
}

// :id -> Id_Orden (entero positivo)
// Id_Operario_Destino: operario activo existente
```

EVIDENCIA THUNDER CLIENT

Prueba 1 — Orden y operario existentes → 200 OK + confirmacion

Thunder Client interface showing a successful PATCH request to `http://localhost:3001/api/carga-trabajo/ordenes/135/operario`. The response is `200 OK` with a JSON body:

```
{
  "ok": true,
  "mensaje": "Orden #135 reasignada a Stevan Wagner",
  "data": {
    "Id Orden": 135,
    "Producto": "Chaquetas ",
    "operario_anterior": 52,
    "operario_nuevo": {
      "Id": 52,
      "nombre": "Stevan Wagner"
    }
  }
}
```

Prueba 2 — Orden inexistente → 404 Not Found

Thunder Client interface showing a `404 Not Found` response for a PATCH request to `http://localhost:3001/api/carga-trabajo/ordenes/14/operario`. The response body is:

```
{
  "ok": false,
  "mensaje": "No se encontró la orden #14"
}
```

Prueba 3 — Operario destino inexistente → 404 Not Found

Thunder Client interface showing a `404 Not Found` response for a PATCH request to `http://localhost:3001/api/carga-trabajo/ordenes/135/operario`. The response body is:

```
{
  "ok": false,
  "mensaje": "No se encontró el operario con id 1"
}
```

2.9 PATCH /api/carga-trabajo/operarios:id/estado

PATCH

/api/carga-trabajo/operarios/:id/estado
Activa o desactiva un operario del sistema.

REQUEST

```
PATCH /api/carga-trabajo/operarios/28/estado HTTP/1.1
Host: localhost:3006
x-api-key: texticode-2026
Content-Type: application/json
```

```

{
  "Estado": "inactivo"
}

// :id -> Id_Usuario del operario (entero positivo)
// Estado: "activo" | "inactivo"

```

EVIDENCIA THUNDER CLIENT

Prueba 1 — Estado valido → 200 OK + confirmación

The screenshot shows the Thunder Client interface. The request is a PATCH to `http://localhost:3001/api/carga-trabajo/operarios/47/estado`. The body is a JSON object: `{ "Estado": "activo" }`. The response is a 200 OK status with a JSON body: `{ "ok": true, "mensaje": "Operario Felipe Rincon actualizado a estado 'activo'", "data": { "Id_Usuario": 47, "Nombre_Completo": "Felipe Rincon", "Estado": "activo" } }`.

Prueba 2 — Estado invalido → 400 Bad Request

The screenshot shows the Thunder Client interface. The request is a PATCH to `http://localhost:3001/api/carga-trabajo/operarios/47/estado`. The body is a JSON object: `{ "Estado": "" }`. The response is a 400 Bad Request status with a JSON body: `{ "ok": false, "mensaje": "Estado inválido. Usa: activo, inactivo" }`.

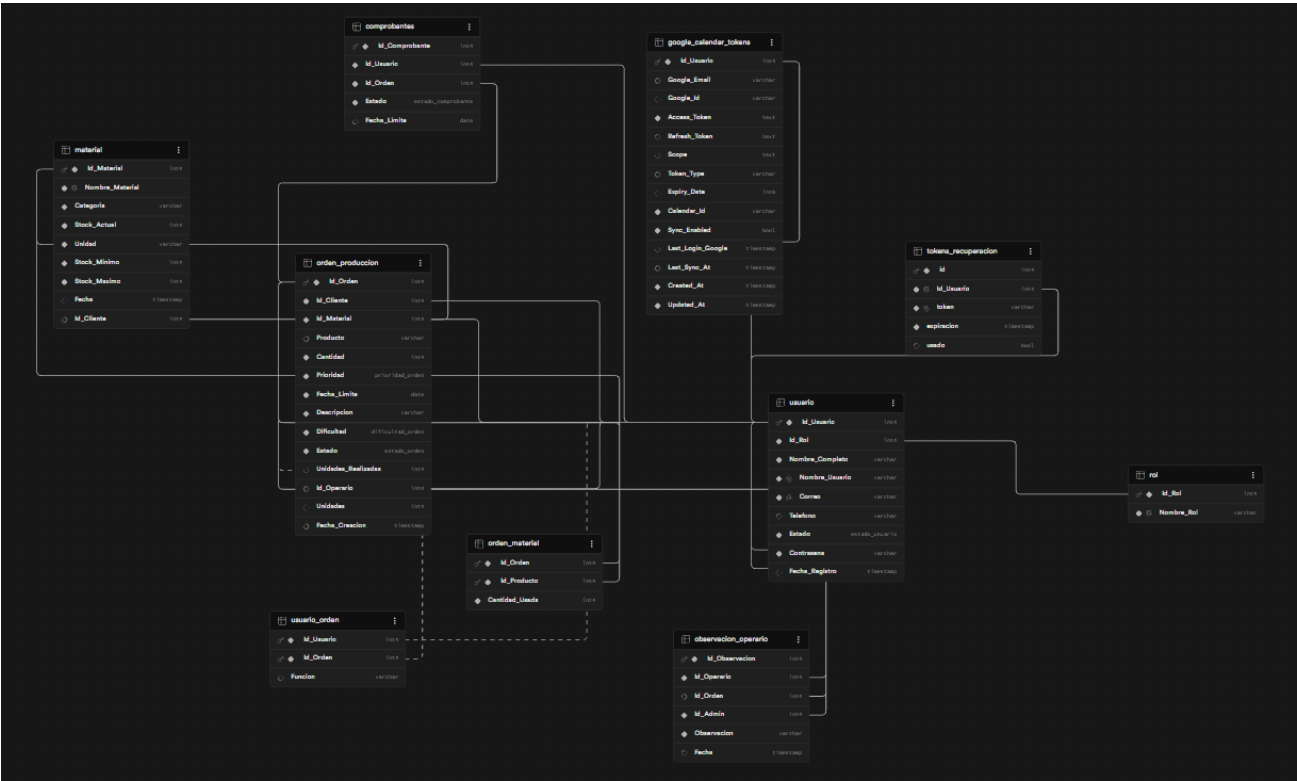
Prueba 3 — Operario inexistente → 404 Not Found

The screenshot shows the Thunder Client interface. The request is a PATCH to `http://localhost:3001/api/carga-trabajo/operarios/1/estado`. The body is a JSON object: `{ "Estado": "activo" }`. The response is a 404 Not Found status with a JSON body: `{ "ok": false, "mensaje": "No se encontró el operario con id 1" }`.

3. Diagrama Entidad-Relación

El módulo de carga de trabajo opera sobre 3 tablas en Supabase PostgreSQL. La tabla central es orden_produccion, que vincula operarios con sus órdenes activas.

Tabla	Campos
usuario	Id_Usuario (PK) · Nombre_Completo · Nombre_Usuario · Correo (UNIQUE) · Telefono · Estado · Id_Rol (FK)
rol	Id_Rol (PK) · Nombre_Rol (admin operario cliente)
orden_produccion	Id_Orden (PK) · Id_Operario (FK) · Id_Cliente (FK) · Id_Material (FK) · Producto · Cantidad · Prioridad · Estado · Unidades · Unidades_Realizadas · Fecha_Limite · Fecha_Creacion



RELACIONES CLAVE

Origen	Relacion	Destino	Descripcion
usuario	1 → N	orden_produccion	Un operario tiene multiples ordenes asignadas
rol	1 → N	usuario	Un rol agrupa multiples usuarios
usuario (admin)	1 → N	orden_produccion	El admin gestiona y reasigna las ordenes

4. Validaciones y Manejo de Errores

Todos los endpoints comparten el middleware apiKey.js y siguen una política uniforme de respuesta de errores.

Escenario	Validacion en codigo	HTTP
API Key ausente	!req.headers['x-api-key']	401 Unauthorized
API Key incorrecta	key !== process.env.API_KEY_EFICIENCIA	403 Forbidden
ID no numerico	isNaN(id)	400 Bad Request
Operario no encontrado	rows.length === 0	404 Not Found
Orden no activa	Estado NOT IN ('En Proceso', 'Pausado')	404 Not Found
Campos requeridos faltantes	!Id_Orden !Id_Operario_Destino	400 Bad Request
Estado invalido en PATCH	!['activo','inactivo'].includes(Estado)	400 Bad Request
Array vacio en multiple	!Array.isArray(movimientos) length===0	400 Bad Request
umbral_sobrecarga < 1	!Number.isInteger(v) v < 1	400 Bad Request
Error interno de BD	catch(err) -> log interno	500 Server Error

MIDDLEWARE APIKEY.JS

```
const verificarApiKey = (req, res, next) => {
  const key = req.headers['x-api-key']
  if (!key)
    return res.status(401).json({ ok: false,
      mensaje: 'Acceso denegado. Se requiere API Key en el header x-api-key' })
  if (key !== process.env.API_KEY_EFICIENCIA)
    return res.status(403).json({ ok: false, mensaje: 'API Key invalida' })
  next()
}
export default verificarApiKey
```